



École Nationale Supérieure d'Informatique
pour l'Industrie et l'Entreprise

First-Year Internship Report

Design of an Internal Dashboard

STUDENT

Omar DAHMAN

ACADEMIC SUPERVISOR

David ROUSSEL

COMPANY SUPERVISOR

Louail POTHIN

COMPANY

RT MOVE



RT MOVE — 26 Rue de la Frette, 78500 Sartrouville, France

July 1 — August 28, 2026

Acknowledgments

I would like to thank my company supervisor, Louail Pothin, for his welcome, his availability, and his advice throughout the internship. I would also like to thank the manager of RT MOVE for allowing me to complete this internship at his company and for trusting me with this project. Thanks as well to my academic supervisor, David Roussel, for his guidance over these two months. Finally, I would like to thank Aurélien, Mounib, Israa, and David, the four other interns, for the good spirit and mutual support that made this teamwork enjoyable and effective.

Abstract

This internship, carried out at RT MOVE (a moving company) from July 1 to August 28, 2026, is part of a digitalization project led by a team of five interns. My assignment (work package L5) consisted of building an **internal web dashboard** to track activity in real time: a map of trucks and jobs, route calculation, live position tracking, sales and marketing indicators, and alerts.

The dashboard was built with Next.js/React/TypeScript, with Leaflet and OpenRouteService for the map, and Supabase for the data. I paid particular attention to making the dashboard **work even when other work packages are not finished**: every live data feed automatically falls back to periodic polling if the live connection is not working, and the link to the shared database was verified through direct tests on the API.

In addition to the dashboard, I also built a **chat assistant** (chatbot) for RT MOVE's public website, suggested by my company supervisor: it answers visitors' questions and registers interested people as real leads, directly in the database.

During the last week of the internship, tests were carried out on all the tools that were built, and a demonstration was given to the manager of RT MOVE to show him how to use the dashboard and the chatbot on a day-to-day basis.

Glossary

Next.js / React JavaScript/TypeScript tools for building websites and web applications.

Supabase A PostgreSQL-based platform that provides a database, a login system, file storage, and live updates.

RLS (Row Level Security) A PostgreSQL rule that restricts access to a table's rows based on the logged-in user.

Realtime Supabase's channel that pushes database changes (`postgres_changes`) live to the browser, without reloading the page.

@supabase/ssr Supabase's official tool that allows the same login session (via cookies) to be shared between the browser and the server.

Polling Periodic checking of the database, used here as a fallback when Realtime is not working.

Leaflet / OpenStreetMap (OSM) A tool for displaying maps, with a free, open map background.

OpenRouteService (ORS) A service that calculates a route and an estimated travel time (ETA).

KPI *Key Performance Indicator*, a key metric used to track performance (sales, marketing).

Gemini Google's language model (artificial intelligence), used for the public website's chat assistant.

Tool calling (*function calling*) A mechanism that lets a language model trigger a specific action (here, saving a lead) instead of simply replying with text.

Streaming response A display mode where the model's answer appears word by word as it is generated, instead of waiting for the full response before showing it.

ADR *Architecture Decision Record*, a document explaining a technical choice and why it was made.

PR (Pull Request) A request to merge a Git branch into the main branch, reviewed by someone else before being accepted.

Milestone (M1–M6) A checkpoint shared by the whole team, at key dates of the internship.

List of Figures

3.1	Login page for the internal area	5
3.2	The four pages of the dashboard	6
3.3	The chat assistant on the public website, after successfully registering a lead	7

Contents

Acknowledgments	i
Abstract	ii
Glossary	iii
List of Figures	iv
1 Introduction: The Company and the Internship	1
1.1 The company RT MOVE	1
1.2 Context of the digitalization project	1
1.3 A team of five interns	1
1.4 My assignment (work package L5) and its objectives	2
2 Background and Useful Concepts	3
2.1 Tools used	3
2.2 Key concepts	3
2.2.1 Web maps and routing	3
2.2.2 Live updates: Realtime vs. polling	3
2.2.3 Data security (RLS)	3
2.3 Setting up the environment	3
3 What Was Built	5
3.1 General architecture	5
3.2 Map and routing	5
3.3 Live tracking and reliability	5
3.4 Automatic refresh	6
3.5 Dashboard alerts	6
3.6 Business indicators (KPIs)	6
3.7 A chat assistant for the public website	7
3.7.1 How it works	7
3.7.2 Technical choice	8
3.7.3 Checks performed and fixes made	8
3.7.4 Organization and status	8
3.8 Integration with the Supabase database	8
3.8.1 Principle and security rule	8
3.8.2 An issue found, then resolved	9
3.8.3 Switching to the real login system	9

3.8.4	Why some tables stayed empty, and what has changed since	9
3.9	Integration with the CRM's codebase	9
3.10	Design overhaul	10
3.11	Verification approach	10
3.12	Difficulties encountered	10
3.12.1	A corrupted build cache	10
3.12.2	Database access not open at first	10
3.12.3	Dependencies between work packages	11
3.12.4	A false alarm about the marketing data	11
3.12.5	Generated text without accents	11
4	Conclusion	12
A	Sustainability and Corporate Social Responsibility	13
B	Technical Details	14
B.1	Live tracking tool with polling fallback	14
B.2	Dashboard alerts	14
B.3	Secure call to OpenRouteService (server side)	14
B.4	Browser-side Supabase client (real login system)	15
B.5	Chat assistant: streaming response with the <code>save_lead</code> tool call	15
	Bibliography and Web References	15

Chapter 1

Introduction: The Company and the Internship

1.1 The company RT MOVE

RT MOVE is a transport, moving, and cleaning company. It operates in the Paris region (Île-de-France) and neighboring regions. To grow, the company wants to improve its digital tools in order to better manage its activity, bring its data together in one place, and automate the search for business clients: that is the purpose of the digitalization project described in this report.

The company is run by two people, registered with the French National Business Registry (RNE), maintained by the INPI:

- **Louail Giovanni POTHIN**, CEO — also my company supervisor;
- **Thomas ROLLOT**, president of the company.

1.2 Context of the digitalization project

Before this project, sales activity and job management were tracked using Excel files (prospecting, sales, job tracking, marketing content). This way of working showed its limits: manual data entry, no live view, and difficulty in bringing the numbers together. The project aims to gradually replace these files with a proper system (database, CRM, mobile app for the field, automations, and a dashboard).

1.3 A team of five interns

The project is split into work packages (L0 to L8) shared among five interns:

- **David** — Technical foundations and automation, automated prospecting and marketing (L0, L6).
- **Aurélien** — Product owner and data, database modeling, the visible part of the CRM, acceptance testing (L1, L3, L8).
- **Mounib** — Backend and security, database, API, security (L1, L2, L4).
- **Israa** — Mobile app for the field, GPS tracking (L7) — *left the internship partway through*.

- **Omar (me)** — Live web dashboard (L5).

Israa left the internship before finishing work package L7. Her part (mobile app, GPS position reporting) was therefore never delivered. This does not call into question the work done on work package L5: the map and the live tracking remain functional and were designed to tolerate an empty `positions_vehicules` table (see section 3.8), but they will not show real positions until another solution has been found for that work package. Following her departure, my scope shifted to focus on the KPI indicators (sales and marketing), and my company supervisor offered me a complementary project, the public website's chat assistant (see section 3.7).

1.4 My assignment (work package L5) and its objectives

My work package consists of building the internal dashboard:

1. a **map** of the day's trucks and jobs (Leaflet + OSM), with routes and estimated travel times (OpenRouteService);
2. **live tracking** of GPS positions sent by the mobile app;
3. **business indicators** (sales funnel, marketing figures);
4. a **daily overview** with alerts (hot lead, overdue job).

Chapter 2

Background and Useful Concepts

2.1 Tools used

The dashboard uses: Next.js 16, React 19, TypeScript, Tailwind CSS 4, and shadcn/ui for the interface; Leaflet and OpenStreetMap for the map; OpenRouteService for the routes; Recharts for the charts; and the Supabase JavaScript client (login, Realtime, RLS) for the data. The chatbot adds Google's Gemini API for generating responses. Everything lives in the same repository as the CRM, whose shared base I reuse.

2.2 Key concepts

2.2.1 Web maps and routing

Leaflet displays an OpenStreetMap background and custom markers (trucks, job pickup/drop-off points). From GPS coordinates, OpenRouteService calculates a road route (a line on the map) and an estimated travel time.

2.2.2 Live updates: Realtime vs. polling

Two approaches allow the map to be updated without reloading the page: a channel that « pushes » changes (Supabase Realtime, which is notified whenever the database changes) or periodic checking (*polling*). The choice, documented in a technical decision record (appendix [B](#)), keeps Supabase Realtime as the main solution, with automatic fallback to polling.

2.2.3 Data security (RLS)

Tables are protected by RLS (*Row Level Security*) rules: only requests sent by a logged-in user (role **authenticated**) can read the data. The dashboard must therefore be logged in to display even a single row (see section [3.8](#)). The public website's chatbot is an exception: since an anonymous visitor is not logged in, writing a lead goes through a private Supabase connection, reserved for the server (see section [3.7](#)).

2.3 Setting up the environment

The development environment (repository, Git conventions, Supabase project, continuous integration) was set up by David (work package L0). Once the project is cloned, local installation and starting

the development server take only a few minutes thanks to the onboarding guide shared by the team.

Chapter 3

What Was Built

3.1 General architecture

The dashboard is organized into four pages under `/admin/dashboard`: *daily overview*, *live map*, *sales KPIs*, and *marketing KPIs*. It sits on top of the CRM's shared base (menu, login guard) built by Aurélien, thanks to a nested layout that adds its own sub-menu. Login happens on a dedicated page (`/connexion`, see figure 3.1), whose session is then shared across the whole internal area.



Figure 3.1: Login page for the internal area

3.2 Map and routing

The map displays the day's trucks and jobs. Addresses are converted into GPS coordinates (geocoding), then OpenRouteService calculates the route and the estimated travel time. Calls to ORS go through a Next.js server route that protects the API key and handles errors (quota exceeded: code 429, service unavailable: code 502).

3.3 Live tracking and reliability

Vehicle positions are read from the `positions_vehicules` table and updated live thanks to Realtime. An important point of the design is **reliability**: if the Realtime channel is not available (network outage, etc.), the interface automatically switches to polling, and a badge shows the connection status (« Live active » / « Degraded mode ») as well as how old the last update is.

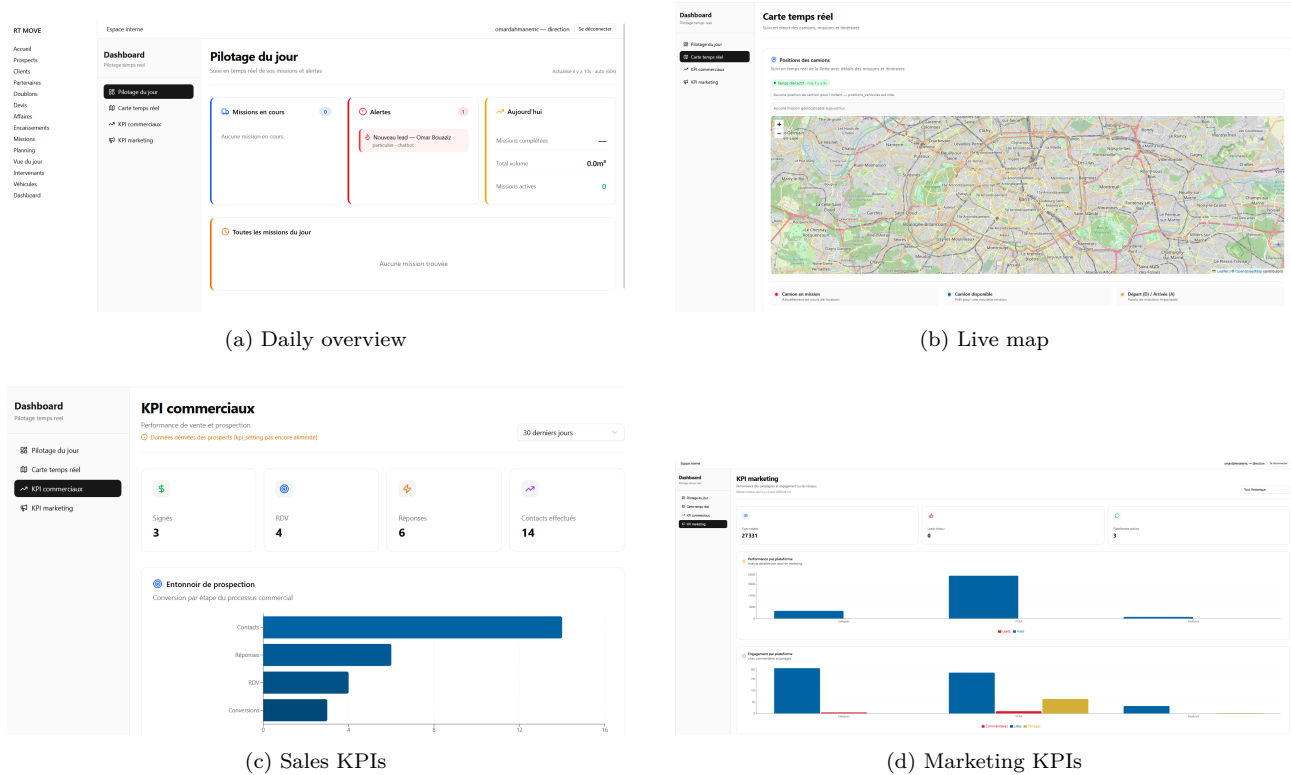


Figure 3.2: The four pages of the dashboard

3.4 Automatic refresh

Three pages used to update only when the page was loaded: *daily overview*, *sales KPIs*, and *marketing KPIs*. They now have an **automatic refresh every 60 seconds**, thanks to a shared tool, with an indicator of the time elapsed since the last update — the same principle already used for the live map’s badge.

3.5 Dashboard alerts

The overview page shows two types of alerts, computed from the real data (see figure 3.2a):

- **Hot lead** — any new lead added to the **prospects** table, whatever its source (contact form, calculator, Leboncoin, or now the public website’s chatbot). David’s scoring tool only keeps leads above a certain score, so a new lead entry corresponds to a lead to flag. The Realtime subscription on new rows shows the alert immediately.
- **Overdue job** — a job whose scheduled date has passed while its status is neither « completed » nor « cancelled ».

3.6 Business indicators (KPIs)

Three families of indicators are provided: a *sales funnel* (contacts → replies → meetings → sales, from `kpi_setting`) and *marketing indicators* (views and leads per platform, from `kpi_marketing`). The

charts are built with Recharts (see the « Sales KPIs » and « Marketing KPIs » pages in figures 3.2c and 3.2d).

A **period selector** (7 days, 30 days, 3 months, or the full history) was added to both pages, using a generic tool shared between the two. Adding it uncovered a **hidden bug**: the sales funnel calculation only took into account the very last KPI row on record, instead of summing all the rows in the selected period; it was fixed at that point.

A second chart, for **engagement** (likes, comments, shares by platform), was also added to the marketing KPI page. It was kept separate from the existing views/leads chart so as not to mix values of very different scales on the same axis, with a color palette checked for readability by colorblind users, using the project’s validation tool.

A freshness indicator (« Last content tracked N days ago », visible in figure 3.2d) was initially designed to show only when there was a delay (more than 3 days without new content). In practice, it turned out to be more useful to display it **at all times**: the display was therefore changed to always show the date of the last content tracked, whether delayed or not.

3.7 A chat assistant for the public website

In addition to the internal dashboard, my company supervisor asked me to build a **chat assistant** (chatbot) for RT MOVE’s public website. The goal is twofold: answer visitors’ common questions (packages, prices, payment, insurance, service area...) and collect the contact details of interested people, to turn them into real leads.



Figure 3.3: The chat assistant on the public website, after successfully registering a lead

3.7.1 How it works

The assistant is a small chat button shown on every page of the public website. Each message sent by a visitor is passed to a language model (*Gemini*, from Google), along with a system prompt containing the real information about the company: the three package options and their prices, payment terms, insurance rules, the service area (Île-de-France and neighboring regions), the company’s contact details, and the founders’ names. This system prompt acts as a guardrail: the assistant must only answer with real information, never invented. Responses are displayed as a stream, word by word, rather than waiting for the full response: the perceived speed is much better, even though the total response time does not change.

When a visitor gives their name and a way to be contacted (phone or email), the assistant can call a **tool** that saves this information directly to the real **prospects** table, the same table used by the other lead sources (Leboncoin listing, contact form, quote calculator) — see figure 3.2a, where a

lead captured by the chatbot appears in the day's alerts alongside the other sources. A check prevents creating a duplicate if the person already exists in the table; if they do, their information is updated instead of being duplicated. Since a public website visitor is not logged in, this write goes through a private Supabase connection, reserved for the server, used only for this specific task.

3.7.2 Technical choice

The *Claude* model (Anthropic) was used first to build the assistant, but its usage is billed from the very first request. The project therefore switched to *Gemini* (Google), which offers a free tier sufficient for testing. A first attempt with a recent Gemini model showed two problems: a very long response time (about 20 seconds) due to an internal reasoning mode enabled by default, and a limit of only 20 free requests per day. The choice therefore landed on a lighter model, which responds in 2 to 3 seconds with a much larger free limit.

3.7.3 Checks performed and fixes made

The behavior was verified directly in the database, not just by looking at the screen: a test conversation was carried out to completion, and the row created in **prospects** was then retrieved with a direct query. These tests surfaced two issues, which were then fixed:

- the registration tool call was not always triggered by the model, even when the conditions were met (name and contact provided); the instructions given to the model were made stricter and more explicit, with a concrete example, which resolved the issue in the tests repeated afterward;
- when a visitor corrected information they had already given (e.g., a phone number), the assistant claimed to have updated their contact details even though the database had not actually been changed; the registration function was fixed to really update the existing row in that case, instead of doing nothing.

A final check focused on spelling: the system prompt had been written without French accents, and the model faithfully reproduced this incorrect spelling in its replies to visitors. The text was rewritten with the correct accents, which fixed the problem (see section 3.12.5).

3.7.4 Organization and status

Since the chatbot is a feature independent from the dashboard (public website versus internal area), its code was kept in its own Git branch (**feat/chatbot-site-public**) and its own *Pull Request*, rather than being mixed in with the L5 work package's.

The assistant was validated by my company supervisor and went live on RT MOVE's public website during the last week of the internship.

3.8 Integration with the Supabase database

3.8.1 Principle and security rule

Each page queries the real Supabase tables directly through dedicated tools (**useMissions**, **useKPICommercial**, **useKPIMarketing**, **useVehiclePositions**, **useAlertes**). Since the tables are protected by RLS for the **authenticated** role only, the dashboard must rely on a valid login session to read even a single row.

3.8.2 An issue found, then resolved

During the first tests, the connection failed with a permission error: requests were being sent anonymously, but RLS only allows logged-in users. By querying **Supabase's API directly** (without going through the application), I confirmed that this was not a bug in my code but a real need for authentication. While waiting for Mounib's real login system, a service account and a temporary access guard (`DashboardAuthGate`) were put in place on the browser side so that work could continue.

3.8.3 Switching to the real login system

Once the real Supabase login system and the database migration were delivered by Mounib, along with the CRM's Quote, Job, and Lead modules delivered by Aurélien, this work was merged into the `feat/dashboard-15` branch (resolving four minor conflicts in shared configuration files). This merge showed that the temporary `DashboardAuthGate` solution had become unnecessary: it logged the dashboard in on the browser side, separately from the real session set by the `/connexion` page (figure 3.1).

The architecture was therefore changed: the browser's Supabase client switched from `createClient` to `createBrowserClient` (the `@supabase/ssr` tool, see the code in appendix B), which lets it **share the same cookies** as the session opened by `/connexion`, with no extra bridging code needed. The service account, the dedicated access guard, and the related environment variables were removed. The full chain was verified: real login → cookies → dashboard pages accessible, with RLS confirmed to allow any logged-in, active user.

3.8.4 Why some tables stayed empty, and what has changed since

Further investigation made it possible to understand why some dashboard pages stayed empty, even though data access was now working:

- the `missions` table was populated either by Mounib's Excel migration (not done yet at the time), or by Aurélien's CRM, which was still using mock data functions instead of the real routes already built on the server side;
- `kpi_setting` had, at that stage, no mechanism at all to keep it fed with data in the code;
- an initial alert on `kpi_marketing` (last data point dated August 7) turned out to be a **false alarm**: the `date` column corresponds to the content's *publication* date, not its collection date, as confirmed by comparing it with the real date of the last Instagram post.

Since then, all three of these points have evolved thanks to the rest of the team's work: Aurélien switched the CRM's missions module to the real server-side API routes (the quote → job flow now creates real rows in `missions`), Mounib imported the real marketing content (Instagram posts and their statistics) into `contenus` and `kpi_marketing`, considerably refreshing the data shown on the marketing KPI page, and he also finished the Excel migration that now feeds `kpi_setting` with real data. All four tables used by the dashboard are therefore now fed with data.

3.9 Integration with the CRM's codebase

My work was built on top of the shared base built by Aurélien (tasks C1 and C2, already merged). This integration required resolving file conflicts: Aurélien's shared `/admin/layout.tsx` layout was

kept (it carries the menu for the whole internal area), while the part specific to the dashboard (side menu, login guard) was moved into a nested layout specific to `/admin/dashboard`.

Later in the internship, the recent deliveries from Mounib (real login system, database migration) and Aurélien (CRM's Quote, Job, and Lead modules) were merged into the `feat/dashboard-15` branch, which required resolving four minor conflicts in shared configuration files (see section 3.8 for details of the login change that this merge triggered). Everything was validated and pushed to GitHub, on a branch intended for a *Pull Request*, as required by the repository's rules (merge reviewed before being accepted).

3.10 Design overhaul

An initial version, with a lot of animation (hover effects, gradients), was reworked toward a more sober style: animations removed, thin-bordered cards, neutral colors, and occasional use of the company's colors (blue `#0063A4`, gold `#D3AF37`).

3.11 Verification approach

Every new feature was verified in several complementary ways:

- type checking with the TypeScript compiler (`tsc -noEmit`);
- a **full production build** (`next build`) after merging in the team's code, to confirm that everything compiles (all CRM and dashboard routes);
- testing the actual call to OpenRouteService with the real API key;
- direct testing of the Supabase API (login, RLS, row counts, add/read/delete) to verify the data chain independently of the interface;
- for the chatbot, test conversations carried out to completion and verified directly in the database, rather than trusting the text shown on screen alone (see section 3.7).

3.12 Difficulties encountered

3.12.1 A corrupted build cache

The local server stopped starting up (running out of memory, then an internal Turbopack error). The cause was not the code but a corrupted `.next` cache folder; deleting it fixed the problem — a good example of the importance of reading the logs before jumping to the conclusion that there is a bug.

3.12.2 Database access not open at first

Described in section 3.8: rather than looking for a workaround on the dashboard side, the approach was to precisely find the real cause (a direct API test) and then make a clear request to the right person.

3.12.3 Dependencies between work packages

A large part of work package L5 depends on deliverables from other interns (Mounib's API, Israa's GPS, David's KPIs). Israa's departure partway through the internship confirmed the value of this approach: the GPS part was in the end never delivered, without that blocking the rest of the dashboard. The main lesson was to build the dashboard so that it would always **remain presentable**, whatever the state of the other work packages (fallback to polling, tolerance for empty tables).

3.12.4 A false alarm about the marketing data

An alert was first raised: the most recent `kpi_marketing` data point seemed to be dated August 7, which could suggest that data collection had stopped. Comparing it with the real date of the last Instagram post showed that the table's `date` column corresponds to the content's *publication* date, not its collection date: the data was in fact up to date, and the alert was a false alarm. This episode is a reminder to always check the exact meaning of a column before drawing a conclusion from it.

3.12.5 Generated text without accents

The chatbot's system prompt had been written without French accents (likely out of habit from working on code, where accents are usually avoided). The language model faithfully reproduced this spelling in its replies to visitors, which gave an unprofessional impression on a tool meant for the public. The fix was simple once the cause was identified: rewrite the system prompt with the correct accents. This incident is a reminder that content meant to be read by real visitors deserves the same proofreading care as the rest of the website, even when it is « just » a configuration text.

Chapter 4

Conclusion

Work package L5 is now complete. The dashboard's four pages (daily overview, live map, sales KPIs, marketing KPIs) are fully functional and connected to the company's real data: the map shows trucks and jobs with their routes, live tracking automatically falls back to polling when Realtime is not available, dashboard alerts surface hot leads and overdue jobs, and sales and marketing indicators are computed on real data with a period selector. The migration to real authentication (`createBrowserClient`) was carried out successfully with the team, and `kpi_setting`, the last table that remained without a data feed, is now also populated thanks to the Excel migration carried out by Mounib: all four tables used by the dashboard are therefore now fed with real data.

In addition to work package L5, the chat assistant suggested by my company supervisor was also delivered: it answers questions from public website visitors and registers interested leads directly in the database, on the same footing as the CRM's other sources.

The last week of the internship was devoted to end-to-end testing of all these tools, followed by a demonstration to the manager of RT MOVE to show him how to use them on a day-to-day basis.

This internship allowed me to work on a real team project, with strong dependencies between the five interns' work packages (and an unexpected event, Israa's departure partway through). Above all, it taught me to build tools that are **robust in the face of uncertainty**: tolerating missing or partial data, planning automatic fallbacks for when an external dependency is unavailable, and always checking directly in the database rather than trusting what is shown on screen alone.

Appendix A

Sustainability and Corporate Social Responsibility

The project contributes to several responsibility objectives:

- **Less paper and double data entry:** the dashboard and the CRM replace manually filled Excel files, which reduces errors and printing.
- **Optimized routes:** route calculation (ORS) and live tracking help reduce empty-running mileage, and therefore fuel consumption and emissions.
- **Open-source tools:** using OpenStreetMap and open-source software reduces dependency on proprietary services.

Appendix B

Technical Details

B.1 Live tracking tool with polling fallback

```
1 const channel = supabase
2   .channel("positions-changes")
3   .on("postgres_changes",
4     { event: "*", schema: "public", table: "positions_vehicules" },
5     () => fetchVehiclePositions())
6   .subscribe((status) => {
7     if (status === "SUBSCRIBED") {
8       setConnectionStatus("realtime"); stopPolling();
9     } else if (["CHANNEL_ERROR", "TIMED_OUT", "CLOSED"].includes(status)) {
10      setConnectionStatus("polling"); startPolling();
11    }
12  });
13 // Fallback if the channel never reaches SUBSCRIBED
14 fallbackTimer = setTimeout(() => { startPolling(); }, 8000);
```

Listing B.1: Simplified excerpt – lib/hooks/use-supabase.ts

B.2 Dashboard alerts

```
1 const today = new Date().toISOString().split("T")[0];
2 // Hot leads: prospects created today
3 const { data: prospects } = await supabase
4   .from("prospects").select("id, nom, prenom, type, source, date_creation")
5   .eq("date_creation", today).order("date_creation", { ascending: false });
6 // Overdue jobs: date passed, status not closed
7 const { data: missions } = await supabase
8   .from("missions").select("id, date_operation, statut")
9   .lt("date_operation", today)
10  .in("statut", ["a_planifier", "planifiee", "en_cours"]);
```

Listing B.2: Simplified excerpt – lib/hooks/use-alertes.ts

B.3 Secure call to OpenRouteService (server side)

```
1 const key = process.env.ORS_API_KEY; // key never exposed to the client
2 const r = await fetch(ORS_URL, { method: "POST", headers, body });
3 if (!r.ok) {
4   return NextResponse.json({ error: `ORS ${r.status}` },
5     { status: r.status === 429 ? 429 : 502 }); // quota vs. unavailable
```

6 }

Listing B.3: Simplified excerpt – app/api/directions/route.ts

B.4 Browser-side Supabase client (real login system)

```

1 import { createBrowserClient } from "@supabase/ssr";
2
3 export function createClient() {
4   return createBrowserClient(
5     process.env.NEXT_PUBLIC_SUPABASE_URL,
6     process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
7   );
8 }
9 // Shares the session cookies set by /connexion:
10 // no more need for a service account or a dedicated auth guard.

```

Listing B.4: Simplified excerpt – lib/supabase/client.ts

B.5 Chat assistant: streaming response with the save_lead tool call

```

1 // Only one tool, no need for a full agent: manual loop over max 3 turns.
2 const chunks = await ai.models.generateContentStream({
3   model: "gemini-flash-lite-latest",
4   contents,
5   config: {
6     systemInstruction: CHATBOT_SYSTEM_PROMPT, // FAQ + real prices from the website
7     tools: [{ functionDeclarations: [SAVE_LEAD_TOOL] }],
8   },
9 });
10
11 for await (const chunk of chunks) {
12   if (chunk.text) controller.enqueue(encodeEvent({ type: "text", value: chunk.text }));
13   if (chunk.functionCalls?.length) functionCalls.push(...chunk.functionCalls);
14 }
15
16 for (const call of functionCalls) {
17   if (call.name === "save_lead") {
18     // Writes or updates the real prospects table, with duplicate checking
19     // (email then phone) before insertion.
20     const result = await createLeadFromChatbot(call.args);
21   }
22 }

```

Listing B.5: Simplified excerpt – app/api/chatbot/route.ts

Bibliography

- [1] Next.js documentation. <https://nextjs.org/docs>
- [2] React documentation. <https://react.dev>
- [3] Tailwind CSS documentation. <https://tailwindcss.com/docs>
- [4] shadcn/ui. <https://ui.shadcn.com>
- [5] Leaflet.js. <https://leafletjs.com>
- [6] OpenStreetMap. <https://www.openstreetmap.org>
- [7] OpenRouteService. <https://openrouteservice.org>
- [8] Recharts. <https://recharts.org>
- [9] Supabase documentation (JS client, Realtime, RLS). <https://supabase.com/docs>
- [10] Gemini API documentation (Google). <https://ai.google.dev/gemini-api/docs>